

α, β -CROWN 을 활용한 외부 PyTorch 모델 강건성 검증

Reliable and Trustworthy Artificial Intelligence · Assignment #4

초록. 본 보고서는 신경망 검증 도구 α, β -CROWN 의 모델 및 설정 구조를 조사하고, 기본 예제에 포함되지 않은 외부 PyTorch 모델을 대상으로 로컬 강건성 검증 파이프라인을 구성한 결과를 정리한다. 과제 3 에서 사용한 Marabou 가 SMT 기반 제약 해결 방식에 가까웠다면, α, β -CROWN 은 bound propagation 과 branch-and-bound 를 결합해 출력 경계의 상·하한을 계산하는 방식이다. 따라서 본 과제에서는 도구의 실행 형식, 모델 지정 방식, specification 작성 방식, 그리고 재현 가능한 실행 환경 구성에 초점을 두었다.

1. α, β -CROWN 모델 및 설정 구조 조사

α, β -CROWN 의 실행은 `abcrown.py --config <yaml>` 형태로 이루어진다. 모델, 데이터셋, 검증 속성, solver 옵션이 하나의 YAML 파일에 정리되므로, 실험 조건을 코드 안에 흩어 두는 방식보다 재현성이 좋다. 저장소의 `complete_verifier/models`에는 toy 모델, CIFAR 계열 CNN/ResNet, VNN-COMP 용 모델, non-ReLU 모델, custom specification 예제가 포함되어 있다. 또한 `complete_verifier/exp_configs`에는 tutorial 예제와 VNN-COMP 벤치마크별 YAML 설정이 분리되어 있어, 모델 종류와 검증 목적에 따라 설정을 수정해 사용할 수 있다.

Marabou 와 비교했을 때 가장 큰 차이는 specification 을 표현하는 방식이다. Marabou 에서는 입력 bounds 와 출력 제약을 코드에서 직접 구성하는 경우가 많았지만, α, β -CROWN 에서는 모델 경로, 데이터 로더, norm, epsilon, timeout, branching strategy 등을 YAML 에 명시한다. 이 방식은 반복 실험에는 유리하지만, 처음 사용할 때는 α, β -CROWN 이 기대하는 key 이름과 커스텀 loader 형식을 맞추는 과정이 필요하다.

2. 외부 모델과 데이터셋 구성

이번 실험에서는 α, β -CROWN 기본 모델 디렉터리에 포함되지 않은 작은 PyTorch MLP 를 직접 생성했다. 데이터셋은 두 개의 2 차원 Gaussian cloud 를 만든 뒤 `[0, 1]` 범위로 정규화한 이진 분류 데이터이다. 모델은 `Linear(2, 16) -> ReLU -> Linear(16, 2)` 구조이며, 입력 차원이 낮고 네트워크가 작아 검증기 설정을 확인하기에 적합하다.

항목	내용
모델 파일	<code>models/toy_binary_mlp.pt</code>
모델 구조	2 입력, hidden 16, ReLU, 2 출력 MLP
데이터셋	<code>data/toy_dataset.csv</code>
데이터 생성	두 개의 Gaussian cloud 를 생성한 뒤 <code>[0, 1]</code> 정규화
커스텀 로더	<code>custom/toy_model_data.py</code>
검증 설정	<code>configs/toy_linf_robustness.yaml</code>
검증 속성	기준 입력 주변 L_{∞} 반경 <code>epsilon=0.05</code> 로컬 강건성

검증 property 는 기준 샘플 주변의 작은 perturbation 영역 안에서 모델의 분류 결과가 유지되는지 확인하는 문제로 설정했다. 이는 이미지 모델의 adversarial robustness 검증과 같은 형태이지만, 본 실험에서는 α, β -CROWN 설정과 실행 흐름을 명확히 확인하기 위해 2 차원 toy 문제로 축소했다.

3. 실행 파이프라인 및 결과 기록

실행 흐름은 `run_all.sh` 와 `test.py` 로 나누어 구성했다. `run_all.sh` 는 과제 3 에서 사용한 방식과 유사하게 conda 환경을 확인하고, `environment.yml` 기반으로 환경을 만든 뒤, `requirements.txt` 를 설치하고 Python 파일의 기본 컴파일을 수행한다. 모델과 데이터

파일이 없으면 `scripts/build_toy_external_model.py` 를 실행해 필요한 산출물을 만든다. 이후 `test.py` 가 YAML 설정을 읽고 α, β -CROWN 실행 커맨드를 구성한다.

현재 저장소 환경에는 α, β -CROWN 본체가 포함되어 있지 않기 때문에, 실제 verifier 실행 단계에서는 `abcrown.py` 를 찾지 못했다. 따라서 `results/verification_result.json` 에는 `environment_not_ready` 가 기록되었다. 이 결과는 모델이나 YAML 설정의 논리적 실패가 아니라, α, β -CROWN 설치 경로가 아직 연결되지 않은 상태를 의미한다. 실제 검증을 수행하려면 다음과 같이 설치된 α, β -CROWN 경로를 지정하면 된다.

```
ABCROWN_ROOT=/path/to/alpha-beta-CROWN ./run_all.sh --run
```

ABCROWN_ROOT 가 올바르게 지정 되면 같은 `configs/toy_linf_robustness.yaml` 을 사용해 `verified`, `falsified`, `timeout`, `unknown` 중 하나의 결과와 실행 시간을 결과 JSON 에 기록하도록 구성했다.

4. Marabou 와의 비교 및 해석

Marabou 는 입력 변수와 출력 제약을 명시적으로 구성하면서 검증 조건을 직접 다룰 수 있다는 장점이 있다 . 작은 네트워크에서는 어떤 제약이 들어가는지 확인하기 쉬웠다. 반면 α, β -CROWN 은 YAML 설정을 중심으로 모델, 데이터, specification, solver 를 묶어 관리하므로 실험 조건을 재사용하기 쉽다. 특히 bound propagation 과 branch-and-bound 를 활용하기 때문에 큰 ReLU 네트워크에서는 Marabou 보다 확장성 있는 검증을 기대할 수 있다.

다만 α, β -CROWN 은 환경 설정의 의존성이 크다. 커스텀 PyTorch 모델을 사용하려면 모델 정의 함수, checkpoint 경로, 데이터 로더, YAML 의 `model/data/specification` 항목이 서로 맞아야 한다. 이번 실험에서 가장 중요한 관찰점도 이 부분이었다. 즉, 검증 알고리즘 자체를 실행하기 전에도 설정 파일과 실행 환경을 정확히 연결하는 과정이 실험 재현성에 직접적인 영향을 준다.

5. 한계 및 결론

본 실험은 외부 모델과 데이터셋을 α, β -CROWN 형식에 맞게 구성하고, 실행 가능한 YAML 및 자동 실행 스크립트를 준비했다는 점에서 과제 요구사항의 구현 부분을 충족한다. 그러나 현재 로컬 환경에서는 α, β -CROWN 본체가 연결되지 않아 실제 `verified/falsified` 결과와 runtime 비교까지는 완료하지 못했다. 제출 전 또는 후속 실험에서는 α, β -CROWN 설치 경로를 ABCROWN_ROOT 로 지정해 결과 JSON 을 갱신하고, 과제 3 의 Marabou 실행 시간과 같은 조건에서 비교하는 것이 필요하다.

결론적으로 α, β -CROWN 은 설정 기반 실험 관리와 대형 ReLU 네트워크 검증 측면에서 강점이 있다. 반면 초기 환경 구성과 YAML 형식 이해가 필수적이므로, 작은 외부 모델로 먼저 파이프라인을 검증한 뒤 더 큰 모델로 확장하는 접근이 적절하다고 판단했다.